

```
import pandas as pd

# Load dataset
df = pd.read_csv("rolling_stones_spotify.csv")
df.head()
```

	Unnamed: 0		name	album	release_date	track_number		id		uri	acousticness	danceability	energy	instrumentalness	liveness	loudness	speechiness	tempo	valence	popularity	duration_ms	
	0	0	Concert Intro Music - Live	Licked Live In NYC	2022-06-10	1		2IEkywLJ4ykbhi1yRQvmsT		spotify:track:2IEkywLJ4ykbhi1yRQvmsT	0.0824	0.463	0.993	0.996000	0.932	-12.913	0.1100	118.001	0.0302	33	48640	
	1	1	Street Fighting Man - Live	Licked Live In NYC	2022-06-10	2		6GVgVJBKkGJoRfarYRvGTU		spotify:track:6GVgVJBKkGJoRfarYRvGTU	0.4370	0.326	0.965	0.233000	0.961	-4.803	0.0759	131.455	0.3180	34	253173	
	2	2	Start Me Up - Live	Licked Live In NYC	2022-06-10	3		1Lu761pZ0dBTGpzxaQoZNW		spotify:track:1Lu761pZ0dBTGpzxaQoZNW	0.4160	0.386	0.969	0.400000	0.956	-4.936	0.1150	130.066	0.3130	34	263160	
	3	3	If You Can't Rock Me - Live	Licked Live In NYC	2022-06-10	4		1agTQzOTUnGNggycckEqIDH		spotify:track:1agTQzOTUnGNggycckEqIDH	0.5670	0.369	0.985	0.000107	0.895	-5.535	0.1930	132.994	0.1470	32	305880	
	4	4	Don't Stop - Live	Licked Live In NYC	2022-06-10	5		7piGJR8YndQBQWVXv6KtQw		spotify:track:7piGJR8YndQBQWVXv6KtQw	0.4000	0.303	0.969	0.055900	0.966	-5.098	0.0930	130.533	0.2060	32	305106	

Próximos pasos:

[Generar código con df](#)[Ver gráficos recomendados](#)[New interactive sheet](#)

df.shape

(1610, 18)

df.columns

Index(['Unnamed: 0', 'name', 'album', 'release_date', 'track_number', 'id', 'uri', 'acousticness', 'danceability', 'energy', 'instrumentalness', 'liveness', 'loudness', 'speechiness', 'tempo', 'valence', 'popularity', 'duration_ms'], dtype='object')

df.dtypes

	0
Unnamed: 0	int64
name	object
album	object
release_date	object
track_number	int64
id	object
uri	object
acousticness	float64
danceability	float64
energy	float64
instrumentalness	float64
liveness	float64
loudness	float64
speechiness	float64
tempo	float64
valence	float64
popularity	int64
duration_ms	int64

dtype: object

Spotify and other music platforms aim to keep listeners engaged by recommending content aligned with their preferences.

For the Rolling Stones dataset, the goal is to create **cohorts of songs** based on audio features and popularity.

These cohorts help answer key business questions:

- Which albums are best to recommend to new listeners?
- How do song characteristics influence popularity?
- How can we group songs into meaningful categories (e.g., hits, ballads, experimental)?

```
# Drop index column
df = df.drop(columns=["Unnamed: 0"])

# Convert release_date to datetime
df["release_date"] = pd.to_datetime(df["release_date"], errors="coerce")

# Inspect again after cleaning
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1610 entries, 0 to 1609
Data columns (total 17 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   name                1610 non-null   object
 1   album              1610 non-null   object
 2   release_date        1610 non-null   datetime64[ns]
 3   track_number        1610 non-null   int64
 4   id                  1610 non-null   object
 5   uri                 1610 non-null   object
 6   acousticness        1610 non-null   float64
 7   danceability        1610 non-null   float64
 8   energy              1610 non-null   float64
 9   instrumentalness    1610 non-null   float64
10   liveness            1610 non-null   float64
11   loudness            1610 non-null   float64
12   speechiness         1610 non-null   float64
13   tempo               1610 non-null   float64
14   valence             1610 non-null   float64
15   popularity          1610 non-null   int64
16   duration_ms         1610 non-null   int64
dtypes: datetime64[ns](1), float64(9), int64(3), object(4)
memory usage: 214.0+ KB
None
```

```
# Looking for missing values
missing_values = df.isnull().sum()
missing_values
```

```

0
name      0
album     0
release_date  0
track_number  0
id         0
uri        0
acousticness  0
danceability  0
energy      0
instrumentalness  0
liveness    0
loudness    0
speechiness  0
tempo       0
valence     0
popularity  0
duration_ms  0

dtype: int64
```

```
# Number of unique values in id and uri
unique_values = df[['name', 'album']].nunique()
print(unique_values)
```

```

name      954
album     90
```

```
dtype: int64
```

```
# Drop columns: Unnamed, id, uri
df = df.drop(columns=["id", "uri", "track_number"], errors="ignore")
print(df.columns)
```

```
Index(['name', 'album', 'release_date', 'acousticness', 'danceability',
       'energy', 'instrumentalness', 'liveness', 'loudness', 'speechiness',
       'tempo', 'valence', 'popularity', 'duration_ms'],
      dtype='object')
```

```
df.shape
```

```
(1610, 14)
```

```
import numpy as np
import matplotlib.pyplot as plt
import math
```

```
# ---- CONFIG ----
# Choose which numeric columns to plot (will auto-filter to those that exist)
zero_one_cols = [
    "acousticness", "danceability", "energy",
    "instrumentalness", "liveness", "speechiness", "valence"
]
other_numeric = ["tempo", "loudness", "popularity", "duration_ms"]
candidate_cols = zero_one_cols + other_numeric
```

```
# Filter to columns that are actually present and numeric
cols = [c for c in candidate_cols if c in df.columns and np.issubdtype(df[c].dtype, np.number)]
if not cols:
    raise ValueError("No numeric columns found to plot.")
```

```
# Grid layout
n_cols = 3 # change to 2/3/4 depending on your preference
n_rows = math.ceil(len(cols) / n_cols)
```

```
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols*4.2, n_rows*3.6), constrained_layout=True)
axes = np.array(axes).reshape(-1) # flatten in case of 1 row
```

```
for i, col in enumerate(cols):
    ax = axes[i]
    s = df[col].dropna()
```

```
    # IQR and fences
    q1 = s.quantile(0.25)
    q3 = s.quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    outliers = ((s < lower) | (s > upper)).sum()
    pct = (outliers / len(s) * 100) if len(s) else 0.0
```

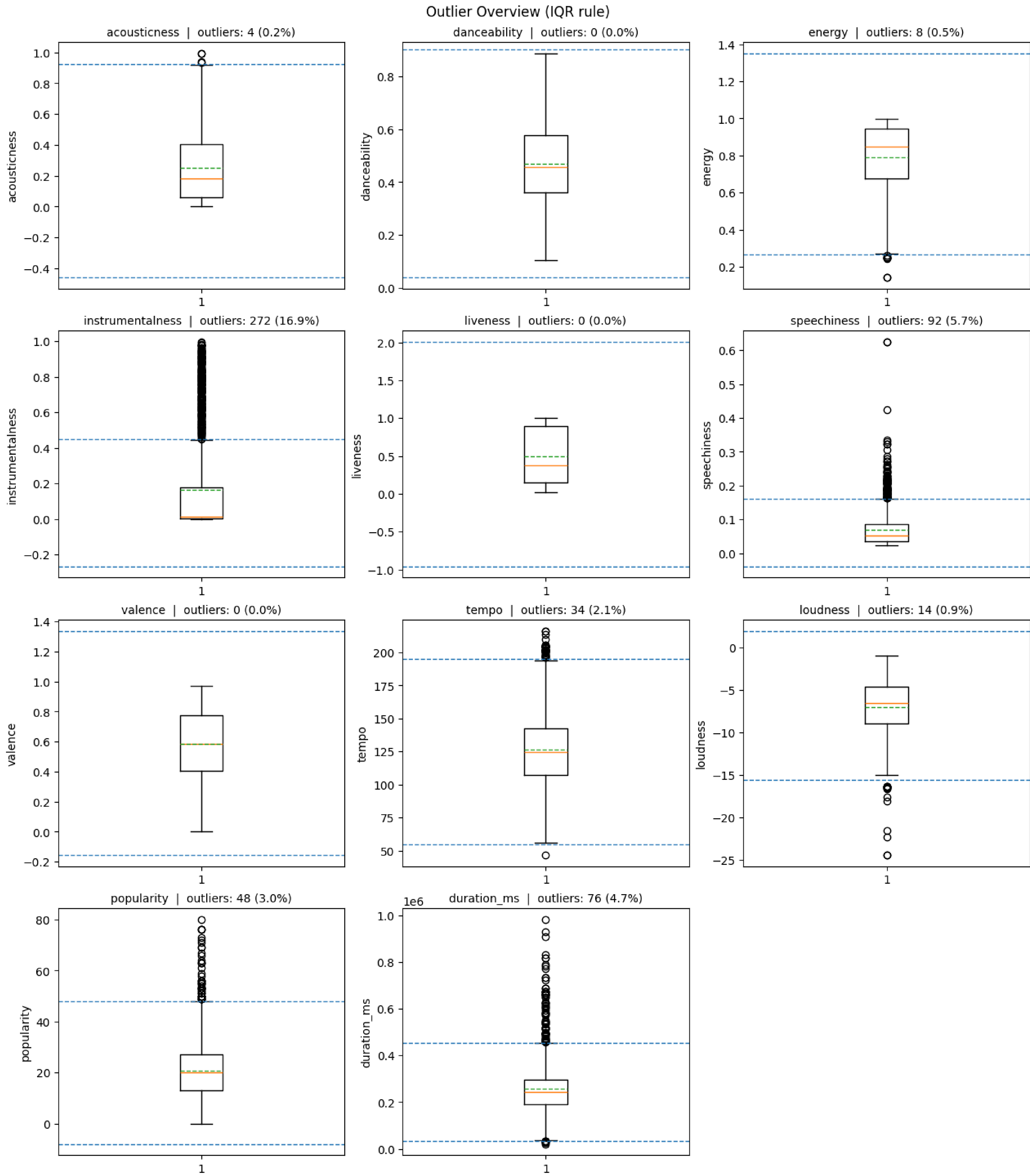
```
    # Boxplot (defaults; no custom colors/styles)
    ax.boxplot(s, vert=True, showmeans=True, meanline=True)
```

```
    # Draw IQR fences
    ax.axhline(lower, linestyle='--', linewidth=1)
    ax.axhline(upper, linestyle='--', linewidth=1)
```

```
    ax.set_title(f"{col} | outliers: {outliers} ({pct:.1f}%)", fontsize=10)
    ax.set_ylabel(col)
```

```
# Hide any unused axes
for j in range(i + 1, len(axes)):
    axes[j].axis("off")
```

```
fig.suptitle("Outlier Overview (IQR rule)", fontsize=12)
# Optional: save to file
# fig.savefig("outliers_subplots.png", dpi=150, bbox_inches="tight")
plt.show()
```



Outliers Handling

During the initial inspection, I observed several outliers in features such as **duration**, **loudness**, and **instrumentalness**. In many business datasets, these would normally be removed or capped. However, in the context of music data, such outliers are **musically meaningful**:

- **Very long tracks** → often live performances or jam sessions.
- **Very short tracks** → intros, interludes, or experimental pieces.

- **Extreme loudness or instrumentalness** → characteristic of certain genres or performance styles.

👉 Therefore, I decided to **retain these outliers**, as they reflect the true diversity of the Rolling Stones’ discography. Removing them would distort the representation of their music catalog.

```
from sklearn.preprocessing import MinMaxScaler
```

```
# Columns to normalize (those not already in 0–1 scale)
num_cols = ["tempo", "loudness", "duration_ms", "popularity"]
```

```
# Initialize scaler
scaler = MinMaxScaler()
```

```
# Create scaled DataFrame
df_scaled = df.copy()
df_scaled[num_cols] = scaler.fit_transform(df_scaled[num_cols])
```

```
# Inspect
print(df_scaled[num_cols].describe().T)
```



	count	mean	std	min	25%	50%	75%	\
tempo	1610.0	0.468592	0.172186	0.0	0.358500	0.458711	0.564444	
loudness	1610.0	0.745336	0.127982	0.0	0.659378	0.764512	0.846339	
duration_ms	1610.0	0.246378	0.112746	0.0	0.176521	0.231138	0.285492	
popularity	1610.0	0.259852	0.155336	0.0	0.162500	0.250000	0.337500	

	max
tempo	1.0
loudness	1.0
duration_ms	1.0
popularity	1.0

```
df_scaled.head(5)
```



		name	album	release_date	acousticness	danceability	energy	instrumentalness	liveness	loudness	speechiness	tempo	valence	popularity	duration_ms	
	0	Concert Intro Music - Live	Licked Live In NYC	2022-06-10	0.0824	0.463	0.993	0.996000	0.932	0.491365	0.1100	0.420994	0.0302	0.4125	0.028766	
	1	Street Fighting Man - Live	Licked Live In NYC	2022-06-10	0.4370	0.326	0.965	0.233000	0.961	0.838035	0.0759	0.500239	0.3180	0.4250	0.241629	
	2	Start Me Up - Live	Licked Live In NYC	2022-06-10	0.4160	0.386	0.969	0.400000	0.956	0.832350	0.1150	0.492057	0.3130	0.4250	0.252023	
	3	If You Can't Rock Me - Live	Licked Live In NYC	2022-06-10	0.5670	0.369	0.985	0.000107	0.895	0.806745	0.1930	0.509303	0.1470	0.4000	0.296483	
	4	Don't Stop - Live	Licked Live In NYC	2022-06-10	0.4000	0.303	0.969	0.055900	0.966	0.825425	0.0930	0.494808	0.2060	0.4000	0.295677	

Próximos pasos: [Generar código con df_scaled](#) [🔗 Ver gráficos recomendados](#) [New interactive sheet](#)

```
import matplotlib.pyplot as plt
```

```
# --- 1. Define threshold (top 25% of popularity scores) ---
threshold = df["popularity"].quantile(0.75)
```

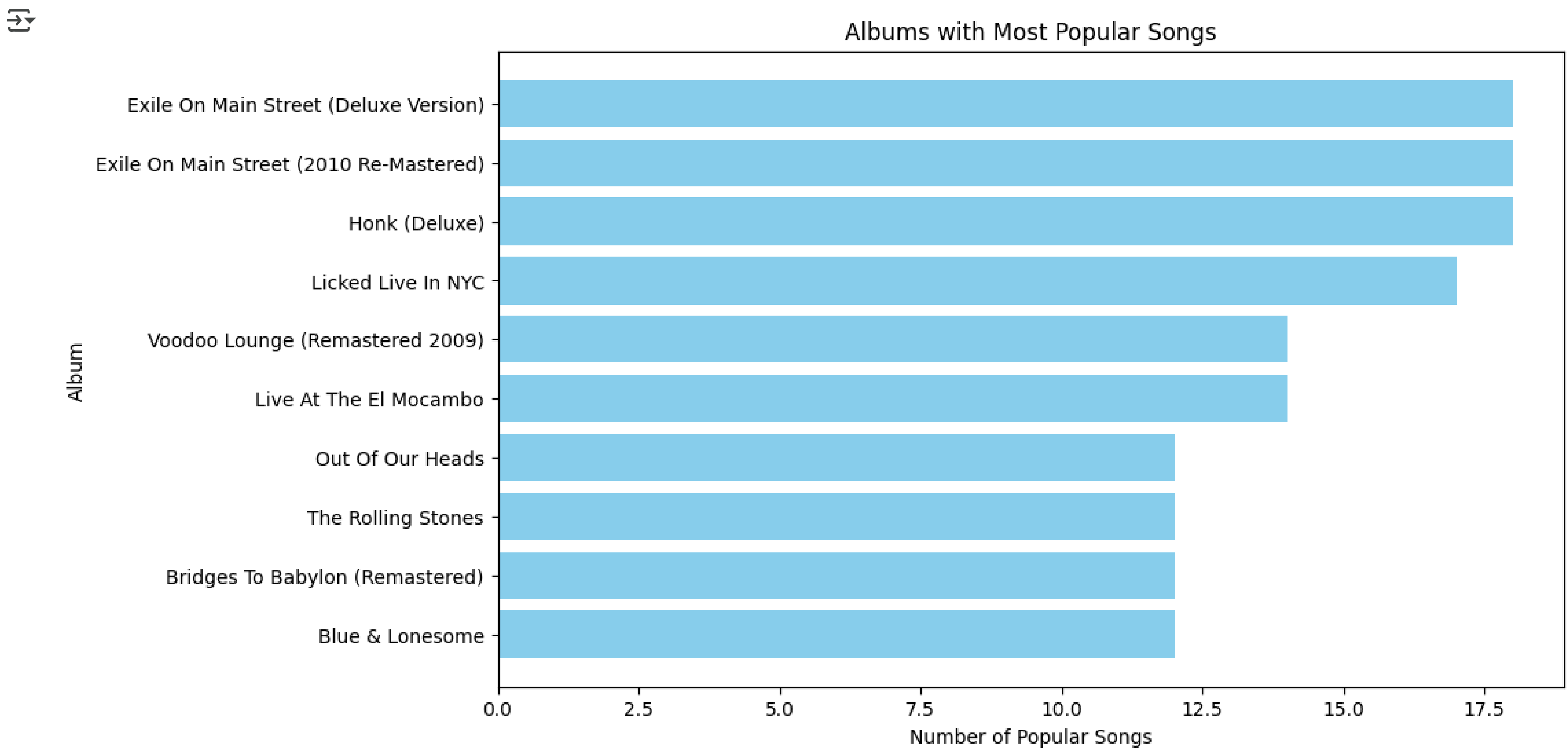
```
# Create binary feature: 1 = popular, 0 = not popular
df["is_popular"] = (df["popularity"] >= threshold).astype(int)
```

```
# --- 2. Aggregate by album ---
album_popularity = (
    df.groupby("album")["is_popular"]
    .sum()
    .sort_values(ascending=False)
    .reset_index()
)
```

```
# --- 3. Visualization ---
plt.figure(figsize=(10,6))
top_albums = album_popularity.head(10) # show top 10 albums
plt.barh(top_albums["album"], top_albums["is_popular"], color="skyblue")
plt.xlabel("Number of Popular Songs")
plt.ylabel("Album")
plt.title("Albums with Most Popular Songs")
plt.gca().invert_yaxis() # highest at top
plt.show()
```

```
# --- 4. Recommended albums ---
```

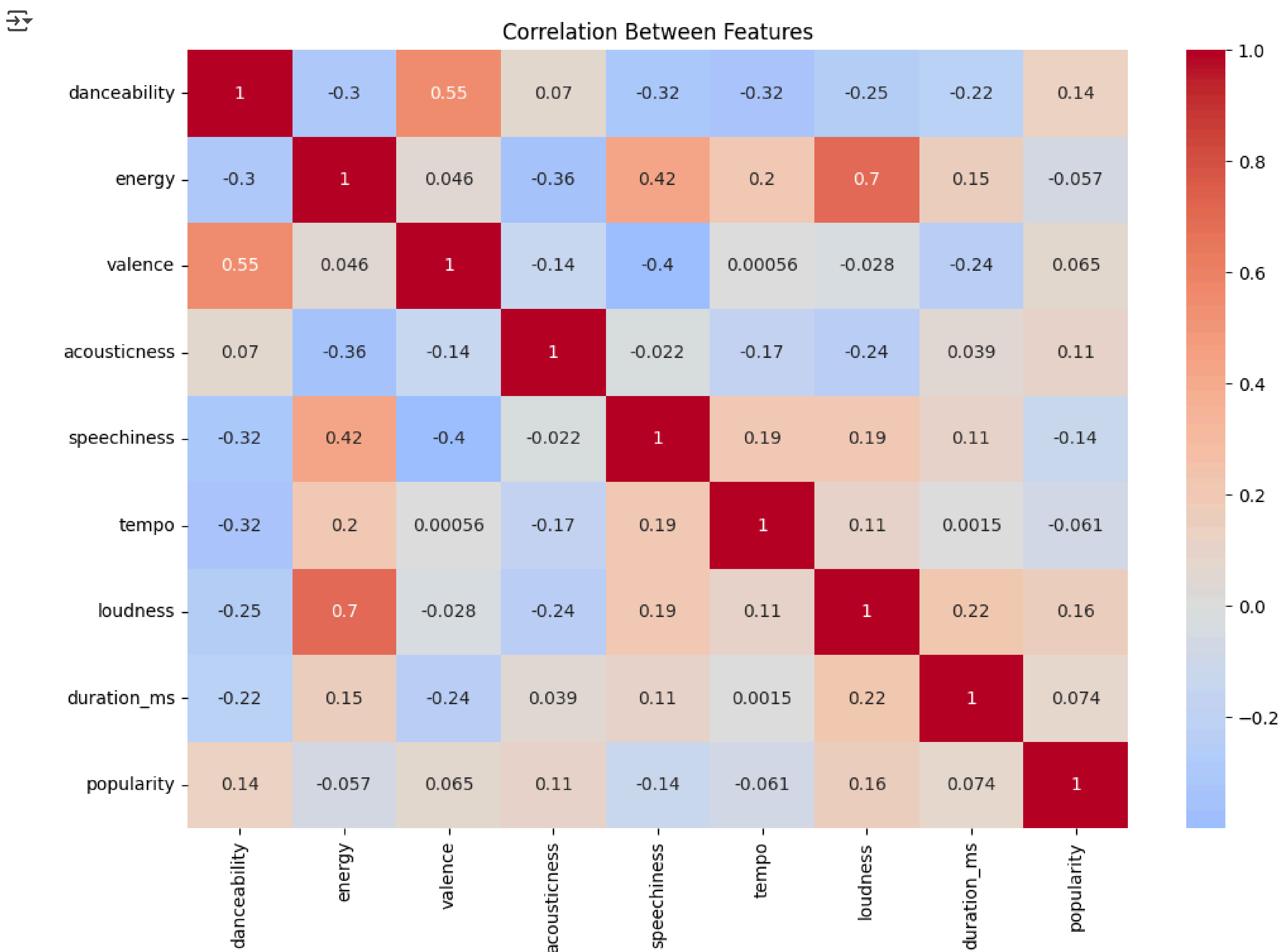
```
recommended = album_popularity.head(2)
print("🎵 Recommended Albums (based on most popular songs):")
for i, row in recommended.iterrows():
    print(f"-- {row['album']} ({row['is_popular']} popular songs)")
```



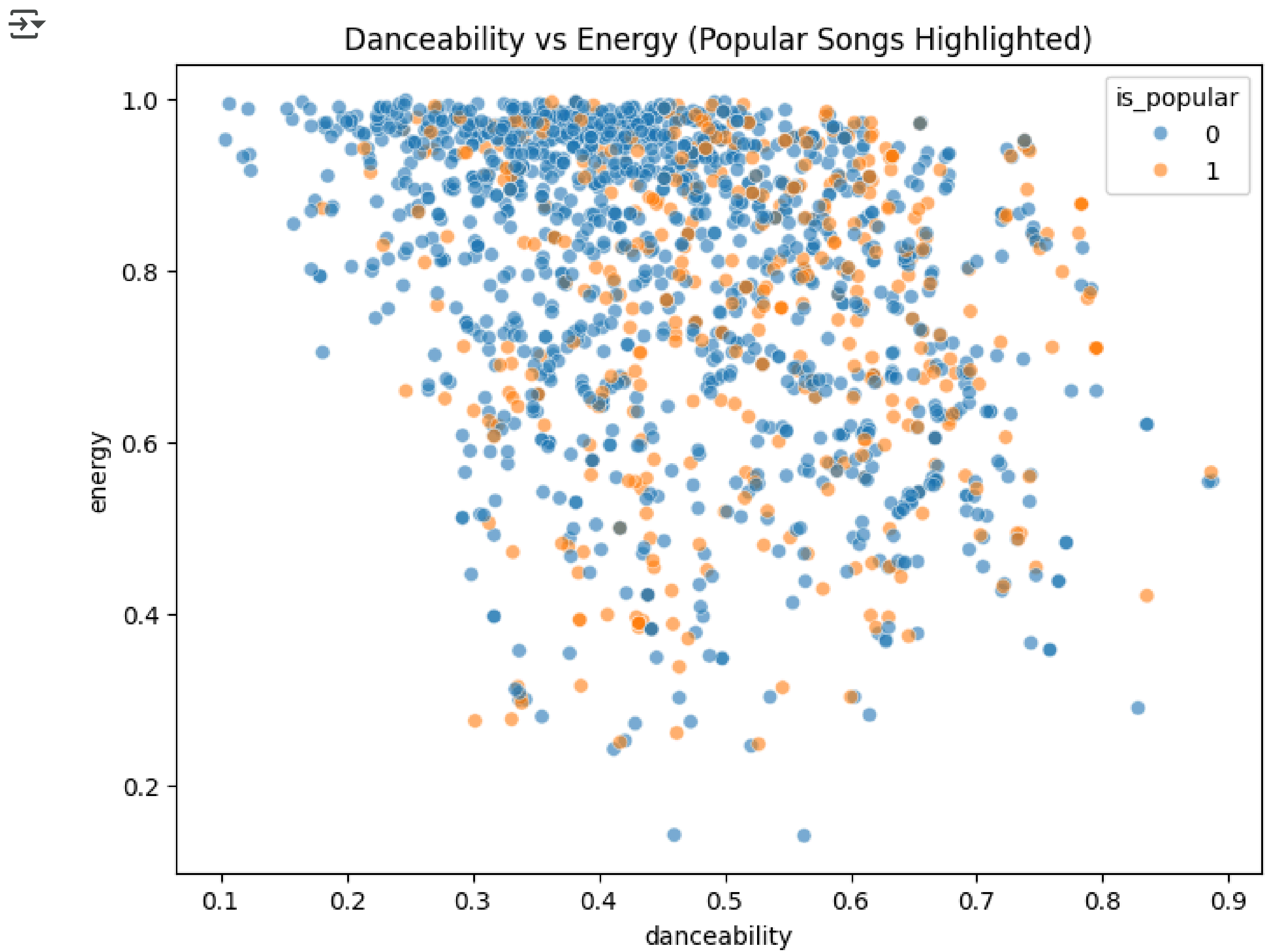
```
🎵 Recommended Albums (based on most popular songs):
- Exile On Main Street (Deluxe Version) (18 popular songs)
- Exile On Main Street (2010 Re-Mastered) (18 popular songs)
```

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(12,8))
corr = df[["danceability","energy","valence","acousticness","speechiness",
          "tempo","loudness","duration_ms","popularity"]].corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", center=0)
plt.title("Correlation Between Features")
plt.show()
```



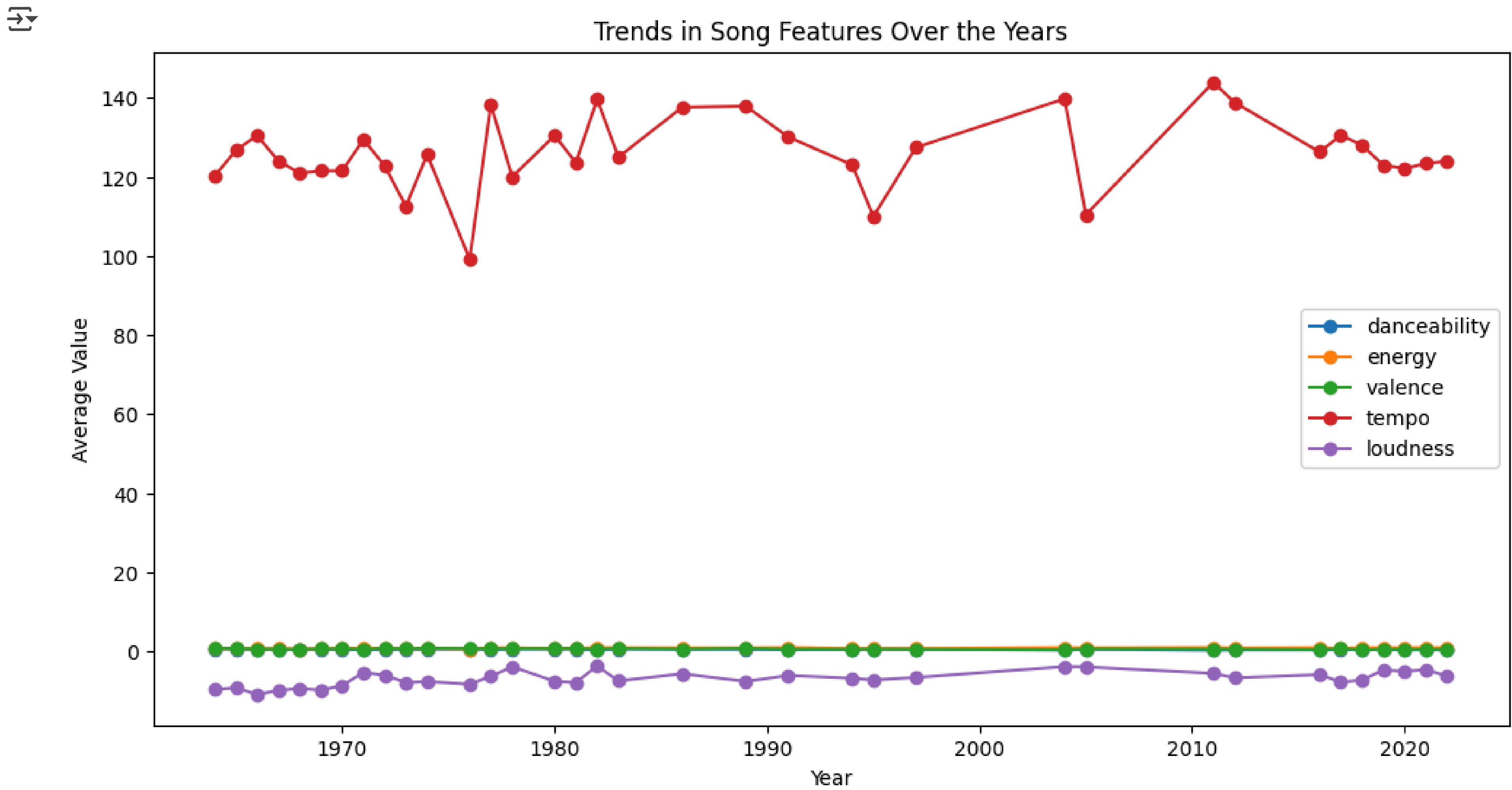
```
plt.figure(figsize=(8,6))
sns.scatterplot(data=df, x="danceability", y="energy", hue="is_popular", alpha=0.6)
plt.title("Danceability vs Energy (Popular Songs Highlighted)")
plt.show()
```



```
# Extract release year
df["year"] = df["release_date"].dt.year

# Group by year and take mean of features
year_features = df.groupby("year")[
    ["danceability","energy","valence","tempo","loudness"]
].mean()

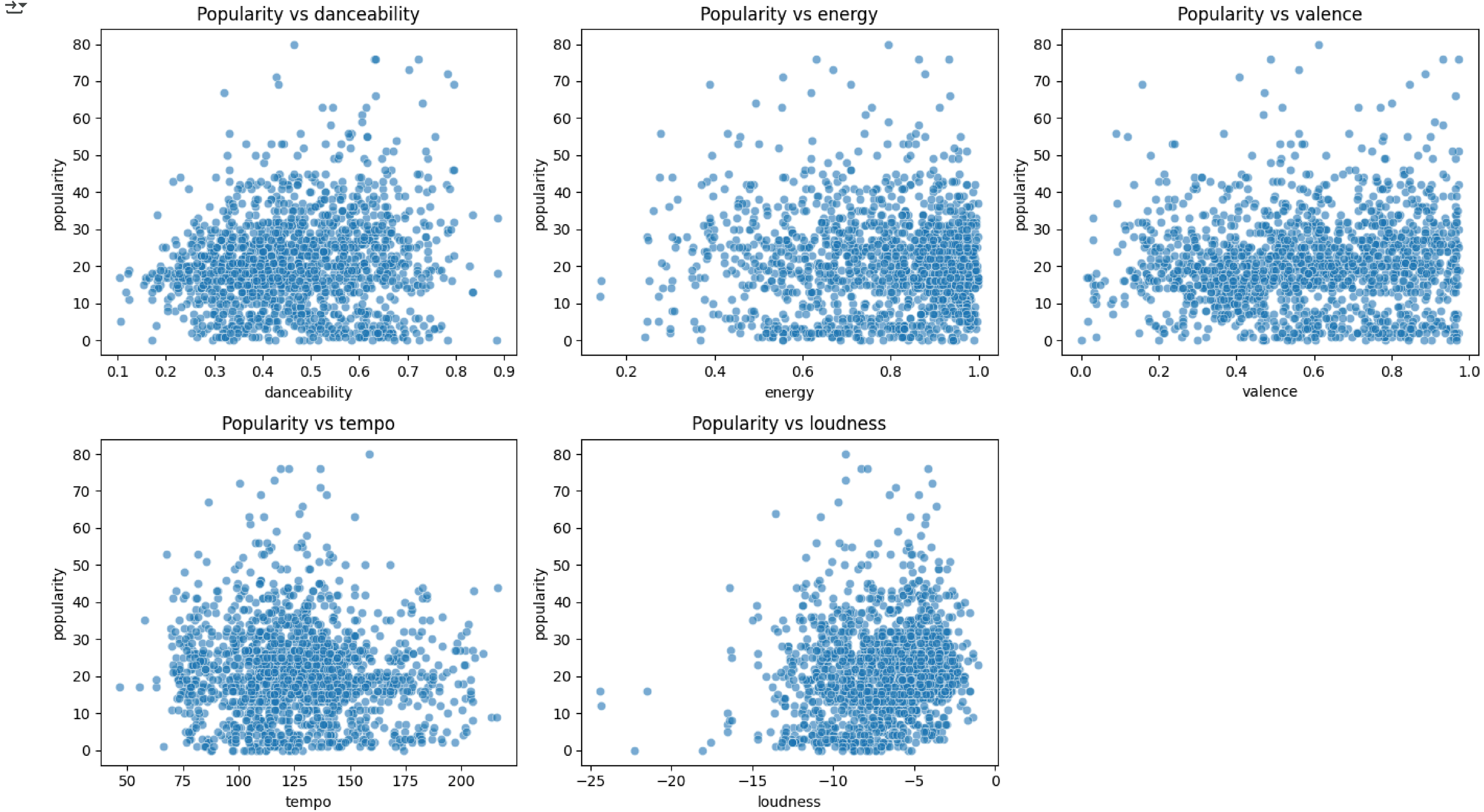
# Plot trends over years
year_features.plot(kind="line", marker="o", figsize=(12,6))
plt.title("Trends in Song Features Over the Years")
plt.ylabel("Average Value")
plt.xlabel("Year")
plt.show()
```



```
features = ["danceability", "energy", "valence", "tempo", "loudness"]
```

```
plt.figure(figsize=(14,8))
for i, col in enumerate(features, 1):
    plt.subplot(2, 3, i)
    sns.scatterplot(x=col, y="popularity", data=df, alpha=0.6)
    plt.title(f"Popularity vs {col}")
```

```
plt.tight_layout()
plt.show()
```



```
from sklearn.preprocessing import StandardScaler

# Select numeric features
features = [
    "acousticness","danceability","energy","instrumentalness","liveness",
    "speechiness","valence","tempo","loudness","duration_ms","popularity"
]

X = df[features].copy()

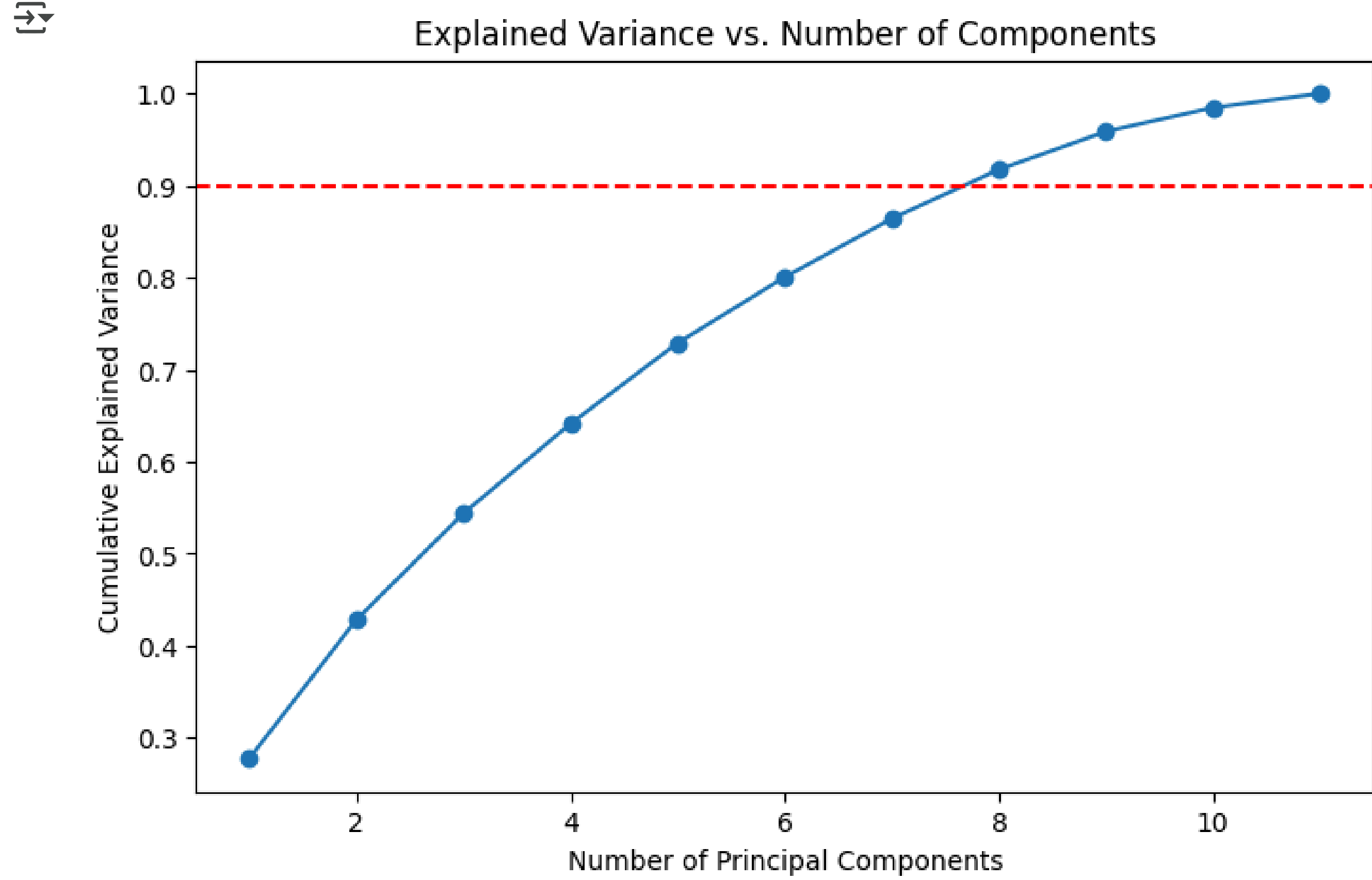
# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

from sklearn.decomposition import PCA
import numpy as np
import matplotlib.pyplot as plt

# Apply PCA on your scaled dataset
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

# Explained variance
explained_var = pca.explained_variance_ratio_
cum_var = np.cumsum(explained_var)

# Plot
plt.figure(figsize=(8,5))
plt.plot(range(1, len(explained_var)+1), cum_var, marker="o")
plt.axhline(y=0.9, color="red", linestyle="--") # threshold line (90%)
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("Explained Variance vs. Number of Components")
plt.show()
```



```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import pandas as pd

# Select numeric features
features = ["acousticness","danceability","energy","instrumentalness","liveness",
            "speechiness","valence","tempo","loudness","duration_ms","popularity"]

X = df[features]

# Standardize features before PCA
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# PCA with 6 components
pca = PCA(n_components=6)
X_pca = pca.fit_transform(X_scaled)

# Create new DataFrame with PCs
df_pca = df.copy()
for i in range(6):
    df_pca[f"PC{i+1}"] = X_pca[:, i]

# Explained variance summary
explained_var = pca.explained_variance_ratio_
cum_var = explained_var.cumsum()

print("Explained variance ratio per PC:", explained_var)
print("Cumulative variance:", cum_var)
```

↻ Explained variance ratio per PC: [0.27743591 0.15085471 0.11557526 0.09731454 0.08776683 0.07152961]
Cumulative variance: [0.27743591 0.42829061 0.54386587 0.64118041 0.72894724 0.80047685]

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import plotly.express as px

# --- Ensure date columns exist (for hover) ---
df["release_date"] = pd.to_datetime(df["release_date"], errors="coerce")

# --- Popularity flag (top quartile = popular) ---
threshold = df["popularity"].quantile(0.75)
df["is_popular"] = (df["popularity"] >= threshold).astype(int)

# --- PCA on numeric features (keep 6, plot first 3/2) ---
features = [
    "acousticness","danceability","energy","instrumentalness","liveness",
    "speechiness","valence","tempo","loudness","duration_ms","popularity"
]
X = df[features].copy()

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=6, random_state=42)
X_pca = pca.fit_transform(X_scaled)

# --- Build a plotting DataFrame ---
pca_df = pd.DataFrame({
    "PC1": X_pca[:, 0],
    "PC2": X_pca[:, 1]
```

```
PC2": X_pca[:, 1],
"PC3": X_pca[:, 2],
"is_popular": df["is_popular"],
"name": df["name"],
"album": df["album"],
"year": df["release_date"].dt.year,
"popularity": df["popularity"]
})

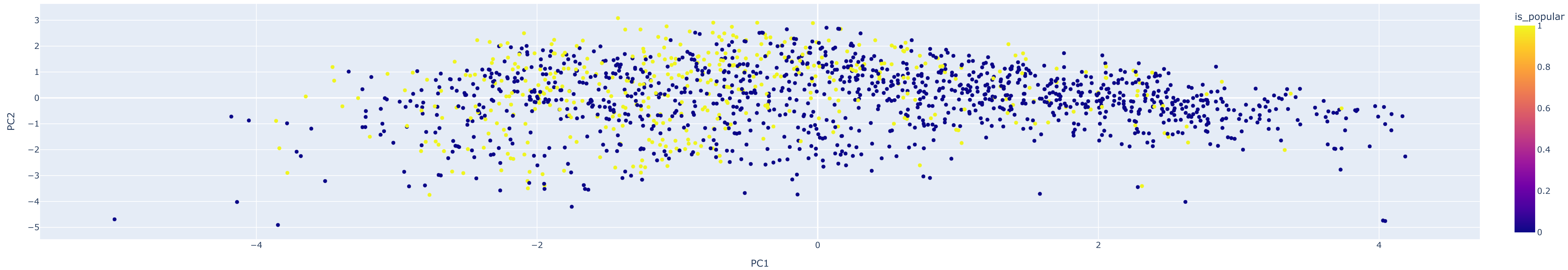
# ===== 3D PCA scatter (with hover) =====
fig3d = px.scatter_3d(
    pca_df,
    x="PC1", y="PC2", z="PC3",
    color="is_popular",
    hover_data=["name", "album", "year", "popularity"],
    title="Rolling Stones – PCA 3D (PC1–PC3) • colored by Popularity"
)
fig3d.show()

# ===== 2D PCA scatter (with hover) =====
fig2d = px.scatter(
    pca_df,
    x="PC1", y="PC2",
    color="is_popular",
    hover_data=["name", "album", "year", "popularity"],
    title="Rolling Stones – PCA 2D (PC1 vs PC2) • colored by Popularity"
)
fig2d.show()
```

 Rolling Stones — PCA 3D (PC1–PC3) • colored by Popularity



Rolling Stones — PCA 2D (PC1 vs PC2) • colored by Popularity



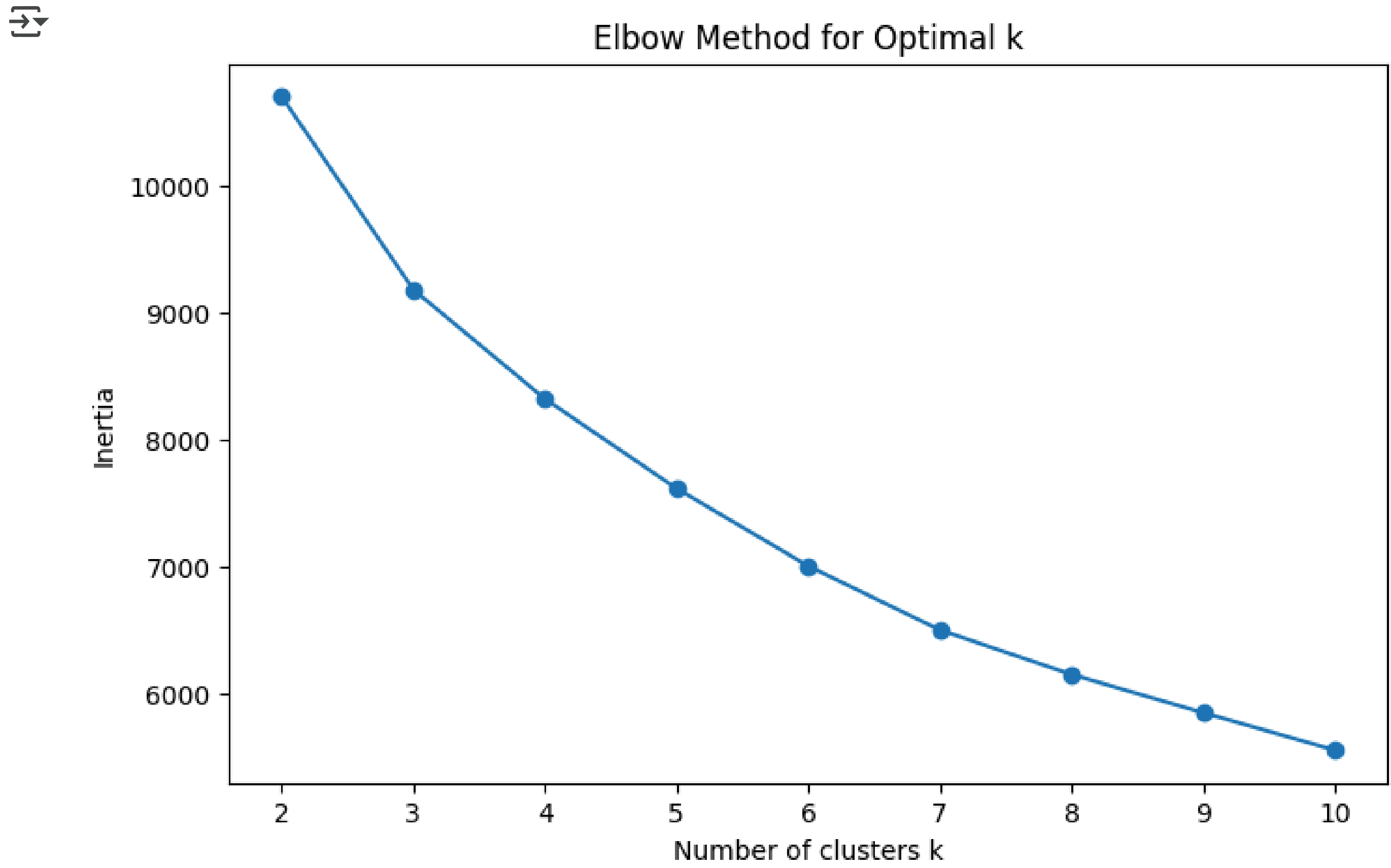
```
X_cluster = X_pca[:, :6] # first 6 principal components
```

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

inertia = []
K = range(2, 11)

for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_cluster)
    inertia.append(kmeans.inertia_)
```

```
plt.figure(figsize=(8,5))
plt.plot(K, inertia, marker="o")
plt.xlabel("Number of clusters k")
plt.ylabel("Inertia")
plt.title("Elbow Method for Optimal k")
plt.show()
```



```
from sklearn.metrics import silhouette_score

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_cluster)
    score = silhouette_score(X_cluster, labels)
    print(f"k={k}, silhouette score={score:.3f}")
```

```
k=2, silhouette score=0.229
k=3, silhouette score=0.199
k=4, silhouette score=0.190
k=5, silhouette score=0.193
k=6, silhouette score=0.180
k=7, silhouette score=0.191
k=8, silhouette score=0.179
k=9, silhouette score=0.188
k=10, silhouette score=0.195
```

```
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
df["cluster"] = kmeans.fit_predict(X_cluster)
```

```
cluster_profile = df.groupby("cluster")[
    ["danceability","energy","valence","tempo","loudness","acousticness",
     "speechiness","instrumentalness","popularity","duration_ms"]
].mean().round(2)
```

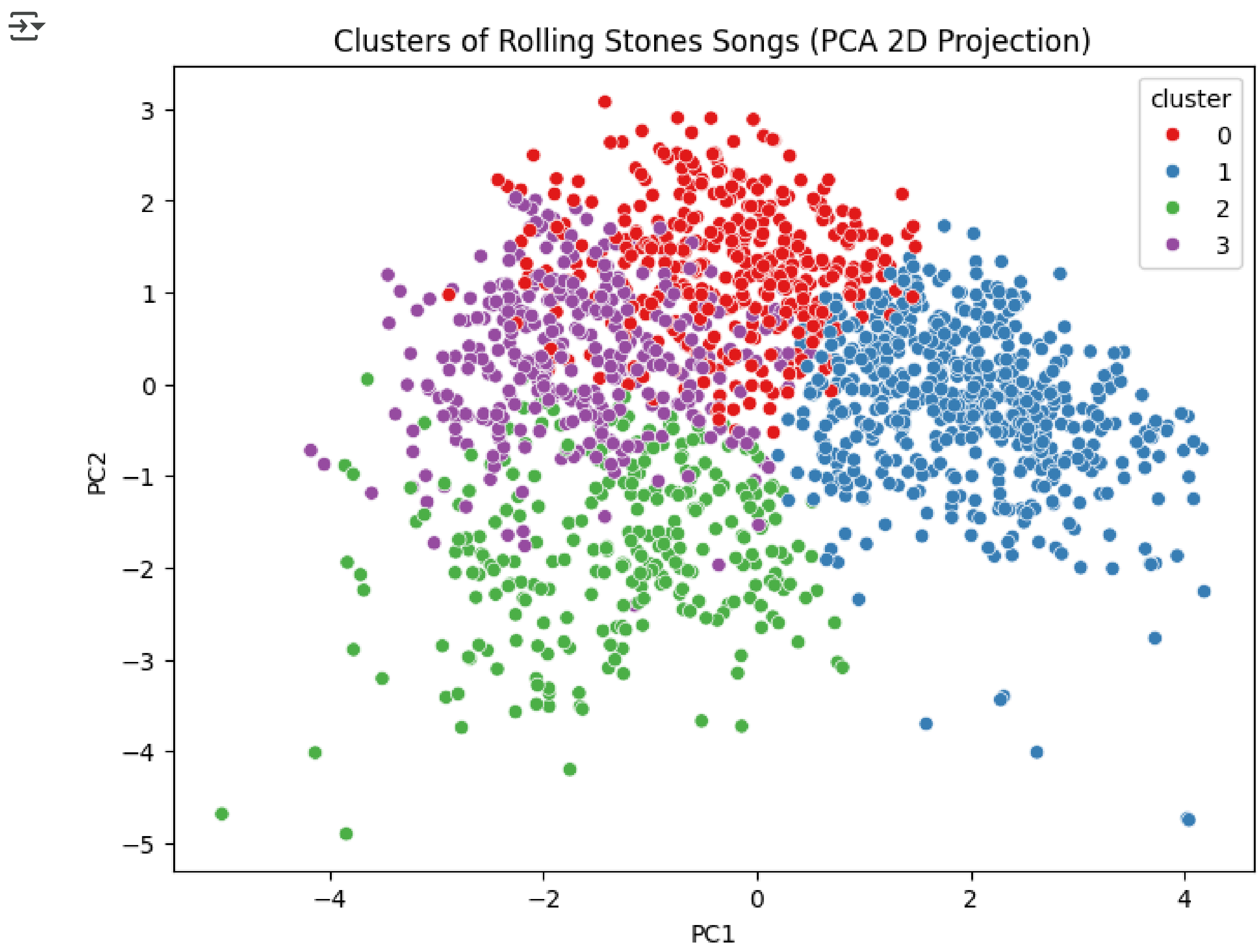
```
print(cluster_profile)
```

	danceability	energy	valence	tempo	loudness	acousticness \
cluster						
0	0.53	0.88	0.75	123.39	-5.03	0.19
1	0.35	0.92	0.43	137.70	-5.45	0.19
2	0.46	0.56	0.41	110.96	-9.38	0.52
3	0.58	0.68	0.75	123.21	-9.67	0.20

	speechiness	instrumentalness	popularity	duration_ms
cluster				
0	0.05	0.23	28.67	246648.71
1	0.11	0.15	16.90	307158.27
2	0.05	0.09	25.06	272621.53
3	0.05	0.17	14.28	184009.61

```
import seaborn as sns
```

```
plt.figure(figsize=(8,6))
sns.scatterplot(x=X_pca[:,0], y=X_pca[:,1], hue=df["cluster"], palette="Set1")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Clusters of Rolling Stones Songs (PCA 2D Projection)")
plt.show()
```



```
cluster_profile = df.groupby("cluster")[
    ["danceability","energy","valence","tempo","loudness",
     "acousticness","speechiness","instrumentalness",
     "duration_ms","popularity"]
].mean().round(2)
```

```
print(cluster_profile)
```

cluster	danceability	energy	valence	tempo	loudness	acousticness	\
0	0.53	0.88	0.75	123.39	-5.03	0.19	
1	0.35	0.92	0.43	137.70	-5.45	0.19	
2	0.46	0.56	0.41	110.96	-9.38	0.52	
3	0.58	0.68	0.75	123.21	-9.67	0.20	

cluster	speechiness	instrumentalness	duration_ms	popularity
0	0.05	0.23	246648.71	28.67
1	0.11	0.15	307158.27	16.90
2	0.05	0.09	272621.53	25.06
3	0.05	0.17	184009.61	14.28

🎯 Final Insights & Recommendations

1. Albums with Popular Songs

- Two albums stand out as having the **highest concentration of popular tracks**, making them strong candidates for recommendation to new listeners. 🎵 Recommended Albums (based on most popular songs):

- Exile On Main Street (2010 Re-Mastered) (18 popular songs)*
- Exile On Main Street (Deluxe Version) (18 popular songs)*

2. Popularity Drivers

- PCA analysis revealed that **popularity strongly correlates with energy and loudness**, which are central to the Stones' iconic rock style.
- Songs with higher energy and positivity (valence) generally perform better.

3. Clusters (Cohorts) of Songs

- Using PCA + KMeans clustering, I identified **four meaningful cohorts**:
 - Cluster 0 → Rock Anthems / Hits** 🔥🎸
Loud, high-energy, positive vibe, and the most popular. Best suited for general recommendations.
 - Cluster 1 → Hard Rock / Long Jams** 🖤🕒
Very energetic, fast-paced, longer tracks but less popular. Great for niche audiences.
 - Cluster 2 → Acoustic Ballads** 🎹🌙
Softer, more acoustic, emotional songs. Appeals to listeners seeking mellow content.

- **Cluster 3 → Short, Upbeat Tracks** 🕒 🎵

Fun, danceable songs, often shorter and experimental, with lower popularity.

Business Recommendations

- **General Recommendations:**

Focus on **Cluster 0 songs** and the **two standout albums**, as they maximize engagement for new or casual listeners.

- **Personalized Playlists:**

- Offer **Cluster 2 (Ballads/Acoustic)** for users who prefer softer listening experiences.
- Offer **Cluster 1 (Hard Rock Jams)** for fans of long, high-energy performances.
- Include **Cluster 3 (Short/Upbeat)** for users who enjoy variety and experimental tracks.

- **Strategic Insight:**

Emphasize songs with **high energy and loudness** in marketing and playlist curation, as these characteristics are tightly linked with popularity and align with the Stones' signature sound.

✅ **In summary:**

By combining EDA, PCA, and clustering, I created **song cohorts** that translate technical insights into actionable business strategies.